

Teaching a Fly Connectome to Predict Text: History, Audit and Limits of a Connectome-Reservoir Character Language Model

Quanxi Li

Independent Researcher

mikey080602@gmail.com

Abstract—We drove a leaky integrate-and-fire model of the complete adult male *Drosophila* CNS connectome (MaleCNS v1.0, 166,700 neurons) with character codes from TinyShakespeare and trained linear softmax readouts to predict the next character. This report records how the project developed, audits its earlier results, and revises the model using mechanisms identified in a companion study. The earlier best frozen model reached 42.55% accuracy and 3.298 bits per character (BPC) on a 20k/5k split; adding dopamine-gated KC→MBON plasticity wrote decodable information into the synapses but did not improve prediction, which motivated the mechanism study. The audit finds that all plasticity results were produced on a simulator version that wrote to the wrong synapses, and that a hashed three-character context head alone, with no brain, reaches 42.9% / 3.205 BPC on our 20k/5k split: the earlier benchmark was essentially an n-gram score. Guided by the mechanism study, sparse projection-neuron codes made the brain improve that hashed context head in all 18 held-out cases (by 0.08 BPC), but the head itself was weak: a Kneser-Ney 5-gram reaches 2.47–2.68 BPC on the same splits, and adding the brain to it makes prediction worse. Three analyses explain why. The brain’s Kenyon-cell code retains the last four or so characters (the character two back is decoded in 57–60% of cases, four back in 24–26%), which a 5-gram already covers; its advantage over the hashed head shrinks from 0.32 BPC at 5k training characters to 0.01 at 100k; and degree-preserving rewired connectomes match the real one within 0.03 BPC. With this protocol and a linear readout the frozen connectome acts as a generic fading memory of a few characters. A GRU trained on the same text reaches 2.34 BPC at 100k characters. In a pilot, a content-dependent dopaminergic teacher, a write gate like a GRU’s, let the mushroom body learn next-character associations online within two or three exposures on a synthetic task (balanced accuracy 26% against 7.8% with a random teacher and 6.7% chance); on real text it also learned, but added nothing to the 5-gram. A small terminal application lets readers compare the models by continuing a prompt.

I. MOTIVATION

The project began as an engineering question: can a real, frozen connectome serve as the dynamical substrate (“reservoir”) of a language model, with only a linear readout trained? A fly has no language circuitry; the aim was never a claim about fly cognition but a test of how much sequence information the wiring can carry and expose, and later of what local plasticity adds.

This language-model project came first. Its central failure, that information written into the mushroom body by plasticity could be decoded from the synapses but did not improve prediction, gave rise to a separate mechanism study [7], which

decomposes where such a memory is stored, what retrieves it and how it reaches downstream neurons. This report closes the loop: it documents the language-model history that led to that study and then uses the mechanisms it found to revise the language model.

II. RELATED WORK

Using the fly connectome as a reservoir is not new. Costi et al. built echo state networks from the most connected neurons of the adult fly connectome and found them more resistant to overfitting in time-series prediction than standard reservoirs [3]. The open project *fly-as-a-lm* trains a 16,384-neuron subgraph of the same MaleCNS v1.0 connectome for character- and token-level text prediction, learning the edge weights by backpropagation while keeping the topology; its character pilot reports 45.21% accuracy and 2.806 BPC on its own alphabet and split [4]. Our model differs in keeping the whole 166,700-neuron connectome frozen, training only a linear readout, and in using local dopamine-gated plasticity rather than gradient learning; the numbers are not comparable. The PN→KC expansion has been described as a locality-sensitive hash that maps similar inputs to overlapping sparse KC codes [5], and sparse KC codes reduce interference between learned associations in an abstract fly model [6]; both anticipate why sparse input helps below.

III. SETUP

Brain. MaleCNS v1.0 [1] simulated with the *flybrain* leaky integrate-and-fire package (20 ms steps, 100 ms membrane time constant), in the spirit of whole-brain executable models [2]. **Input.** Each of the 65 characters is a fixed random set of input channels injected for K steps per character. **Readout.** Spike traces or voltages of selected neurons, standardised, feeding a ridge or softmax classifier for the next character. **Corpus.** TinyShakespeare (1,115,394 characters, 65 symbols). **Metrics.** Validation accuracy and BPC, with unigram and bigram baselines.

IV. HISTORY

Fig. 1 summarises the stages.

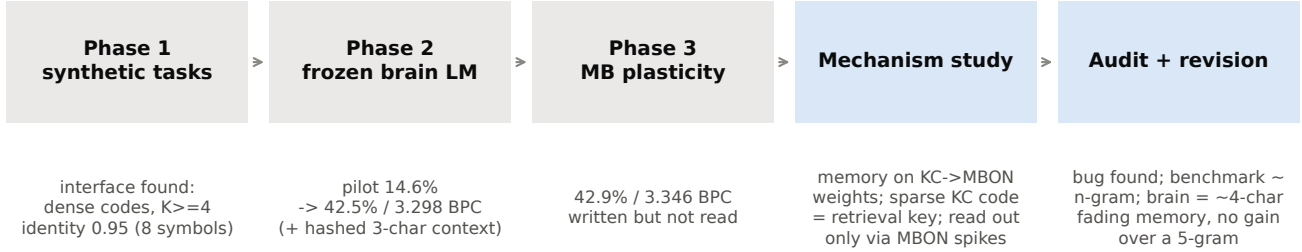


Fig. 1. Project timeline. Gray: the language-model project (Phases 1–3); blue: the mechanism study it led to [7] and the audit and revision reported here. Numbers are read from the metrics files.

A. Phase 1: finding a usable interface (synthetic tasks)

Early experiments on fixed-rule letter tasks failed to generalise (validation at chance, training 0.81–0.92). Single-cell injection moved about 6 of 1,314 descending neurons; groups of 165 input cells moved about 30. Denser codes and sustained injection made the symbol identity decodable across unseen noise seeds (8 symbols: 0.48 with 32 active channels, 0.95 with 512), a deterministic cycle task generalised (0.89, chance 0.33), and a recall task showed a clean memory window of about two tokens.

B. Phase 2: TinyShakespeare with a frozen brain

A first pilot with two steps per character reached only unigram level (14.6%). Grids over steps per character, code density, input windows, position codes, readout populations (descending, VNC, central neurons ranked by a delayed-memory probe), voltage versus trace features, softmax readouts and an explicit hashed order-3 character context produced the frozen reference: 512 memory-ranked central neurons plus a 32,768-bucket context head, 42.55% / 3.298 BPC on a 20k/5k split (the project summary quotes 42.71% / 3.227; the model’s metrics file records the former).

C. Phase 3: adding mushroom-body plasticity

The author proposed using the connectome’s own KC→MBON synapses as a slow memory and optimised this path over several rounds: a plastic lane on the 61,210 KC→MBON edges gated by simulated dopaminergic activity, first read directly and then through small lag decoders fused with the frozen central lane. The synaptic state was decodable, but prediction did not improve: the best fused model reached 42.88% / 3.346 BPC, against 42.55–42.71% / 3.227–3.298 for the frozen reference, and direct plastic readouts were worse (22.4% / 9.00 BPC). This “written but not read” gap is what the companion mechanism study [7] set out to explain.

D. Roles

By the author’s account: the research direction; the probe algorithm and its initial parameter counts; the move from dense whole-network injection and readout, which was slow and noisy, to sparse probe sets, which reduced the noise;

the memory-ranked readout; the experimental parameters and scale of Phases 1–2; and the proposal to route memory through the KC–MBON synapses were the author’s. OpenAI Codex wrote the hashed context head, rewrote training scripts for different parameter and probe-count settings and managed parallel training runs. (The “sparse probes” of Phase 1 refer to how many channels were driven and read; at the Kenyon-cell level the resulting 512-channel codes were still dense, as the audit shows.) From Phase 3 on, the work moved beyond what the author could direct alone: the unexplained “written but not read” result was pursued with a large set of AI agents, which also introduced errors that the audit below corrects. The project log records the author’s instructions but not what proposed each idea; this division is an author statement.

V. AUDIT OF THE EARLIER RESULTS

- **Wrong-synapse writes.** Every plasticity language result (all `plastic_*`, `pilot*` and `overnight_*` runs) predates the 2026-09-17 simulator fix after which plastic writes land on the intended synapses [7]. Frozen results are unaffected.
- **The context head carries the score.** A hashed order-3 context table initialised from smoothed counts, with no brain features, reaches 42.9% / 3.205 BPC on our 20k/5k split. The earlier frozen reference (42.55% / 3.298) is therefore at the level of a pure n-gram model. Our split differs from the earlier multi-stream sampling, so this comparison is approximate. Consistently, the earlier same-brain model fell from 43.1% to 22.9% when the context head was removed (a plasticity run, hence also produced before the simulator fix).
- **Dense input.** All earlier runs used 512 input channels per symbol. The mechanism study found that such codes activate most KCs for every symbol (87% at the probe; overlap between symbols 0.93), which removes the sparse code the mushroom body relies on.

VI. MECHANISM-GUIDED REVISION

Three changes follow from the mechanism study: (1) sparse PN codes (160 or 192 of 675 PNs, drive 1.5), where KC codes are sparse and symbol-specific; (2) readouts that include the

TABLE I

EXPLORATORY SINGLE RUNS OF THE REVISED MODEL (ONE SEGMENT, ONE CODE DRAW; SETTINGS CHOSEN ON THE VALIDATION SPLIT). BRAIN FEATURES: KC COUNTS (4,064), MBON VOLTAGE AND SPIKES (97 EACH), 256 MBON-TARGET CENTRAL NEURONS (TRACE AND VOLTAGE).

Model	Accuracy	BPC
<i>20k/5k (bigram 27.5% / 3.624; unigram 4.736 BPC)</i>		
Earlier frozen reference (dense input)	42.55%	3.298
Context head only (no brain)	42.9%	3.205
Sparse 160: KC code + context	43.2%	3.146
Sparse 160: all features + context, $L2\ 5 \cdot 10^{-3}$	42.4%	3.110
Sparse 160: all features, no context	38.8%	3.229
Sparse 192: all features + context	42.2%	3.217
Sparse 192 + plasticity, teacher every token	40.2%	3.450
<i>6k/2k (bigram 29.3% / 3.747)</i>		
Context head only	42.8%	3.378
Dense 512: all features + context	41.1%	3.374
Sparse 192: all features, no context	43.0%	3.178
Sparse 192: all features + context	43.2%	3.147

sparse KC code and the subthreshold MBON voltage, since downstream spikes carry stored state only when a threshold flips; (3) plasticity only with a teaching signal, since without dopaminergic activity nothing is written. Each character is injected alone (no input window), so any history must come from network dynamics. The readout is a softmax whose hashed context table is initialised from smoothed counts on the training rows; temperature is set on the last 10% of the training rows; one continuous stream; train = the first N characters, validation = the next M .

A. Exploratory runs

Table I lists the exploratory runs. They suggested that sparse input lets the brain improve on the context head, but they came from one segment and one code draw, with the feature set and L2 strength chosen by looking at the validation split. The readout fit is also noisy: refitting one model on the same brain features with four mini-batch orders gives 3.120–3.180 BPC (s.d. 0.026), comparable to the apparent gains. Plasticity with a PAM pulse on every character wrote the synapses (modulation up to 0.17) but made prediction worse, as expected from the mechanism study, where a teaching signal that is the same for every item writes changes that are not item-specific.

B. Held-out check over segments and code draws

We therefore repeated the frozen runs for three PN codes (sparse 160, sparse 192, dense 512) on three text segments (starting at characters 0, 300,000 and 600,000; 20k train / 5k validation each) with three random PN-code draws, 27 simulations in all. For each, the readout (feature set: all, KC only, or MBON and central only; $L2\ 5 \cdot 10^{-3}$, 10^{-2} or $3 \cdot 10^{-2}$) was chosen on the last 10% of the training characters and evaluated once on validation, together with the context head alone on the same split (Fig. 2).

With sparse input the brain lowered BPC in every case: by 0.078 ± 0.020 (160 PNs) and 0.076 ± 0.014 (192), mean $\pm$ s.d. over 9 cases (context head alone 3.097 BPC on average; brain plus context 3.019 and 3.021). With the earlier dense

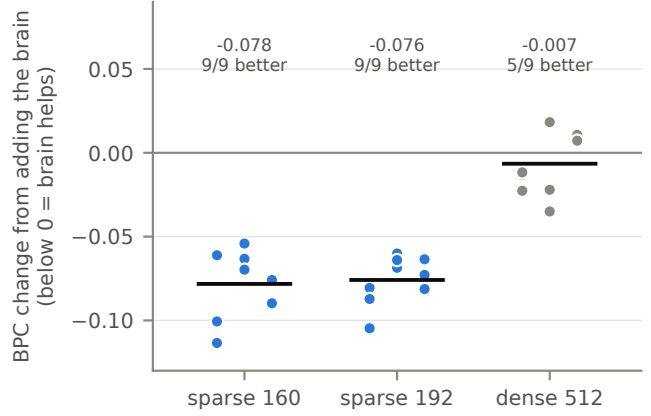


Fig. 2. Change in validation BPC when the brain is added to the context head, per text segment (left to right within a group: offsets 0, 300k, 600k) and PN-code draw. Readouts chosen on held-out training data. Bars: means.

input the change was -0.007 ± 0.019 , an increase in 4 of 9 cases. Because every sparse case improves, the effect is larger than the readout-fit noise above. Accuracy, however, did not improve: 44.6% and 44.5% with the sparse brain against 44.5% for the context head alone, and 43.5% with dense input, lower in all 9 cases. The brain therefore makes the probabilities better calibrated rather than changing the most likely character. On its own the sparse brain is far weaker than the n-gram (33.3% / 3.50 BPC; dense 27.6% / 3.85).

The held-out selection preferred MBON and central features over KC counts in 17 of 18 sparse cases. This turned out to be a preprocessing artefact. Most KCs fire in only a handful of training characters, so standardising their counts produces huge values when they fire at validation ($|z| > 20$ in 15% of validation rows). In an ablation on the three sparse-160 segments with fixed L2 ($5 \cdot 10^{-3}$), KC counts alone lowered BPC by 0.094 relative to the context head, and by 0.107 when the standardised values were clipped at ± 3 . MBON and central features together gave 0.073, the subthreshold MBON voltage alone 0.041 (same direction in all three segments). The KC code thus carries at least as much of the brain’s contribution as the downstream neurons. In the sparse regime the MBONs never fired during the language-model runs, so the central neurons did not receive mushroom-body output; their contribution comes from other pathways.

C. More data

We simulated 120k characters (segment 0, one code draw) and trained on the first 5k to 100k, always validating on the same last 20k characters (Fig. 3; clipped KC features, $L2\ 5 \cdot 10^{-3}$). The brain’s gain over the context head shrank steadily with training size: -0.32 , -0.20 , -0.10 , -0.04 and -0.01 BPC at 5k, 10k, 20k, 50k and 100k characters. At 100k the context head alone reached 48.9% / 2.681 BPC and the brain added 0.010 BPC, within the readout-fit noise, while lowering accuracy slightly (48.4%); the held-out-selected readout was worse than the context head (2.698), and dense KC features

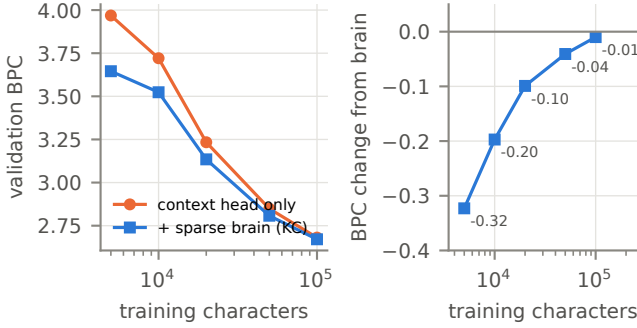


Fig. 3. Learning curve on one 120k-character segment, validated on its last 20k characters. Left: validation BPC with the context head alone and with the sparse brain’s KC features added. Right: the difference.

much worse (2.996). The brain’s advantage is therefore a small-data effect.

VII. WHAT DOES THE BRAIN PROVIDE?

The gains above were measured against the hashed context head. We asked what the brain supplies and tested three predictions of one hypothesis: that the sparse brain is a nonlinear, fading memory of the last few characters, useful only while n -gram statistics are sparse. If so, (P1) the character k steps back should be decodable from the brain only for small k ; (P2) a properly smoothed n -gram of sufficient order should leave the brain nothing to add; and (P3) rewiring the connectome while keeping degrees should not matter. All analyses use the three 20k/5k segments of the held-out check (sparse 160 PN, default code draw) with clipped KC features ($L2\ 5 \cdot 10^{-3}$).

Stronger baselines (P2): We fitted interpolated modified Kneser-Ney (KN) character models [8], [9] on the same training characters. They are far stronger than the hashed head (Table II): a 5-gram reaches 2.47–2.68 BPC, against 3.10 at best for any brain model on these splits. To ask whether the brain adds anything to such a model, we used the KN log-probabilities as a fixed offset in the softmax readout and added brain features (KN probabilities for training rows were cross-fitted over five folds, so that the readout does not learn to over-trust them). With a trigram the KC features helped a little (-0.02 to -0.05 BPC), less than an explicit one-hot code of the last four characters (-0.05 to -0.08); with a 5-gram or 7-gram they made prediction worse ($+0.03$ to $+0.06$; all brain features $+0.10$ to $+0.15$), as extra features do when they carry no new information (Fig. 4, right). A causal cache of recent character statistics, the simplest online memory, did not help the 5-gram either (-0.006 to $+0.012$). A one-layer GRU [10] trained on the same characters reached 2.64–2.98 BPC with 20k characters and 2.34 with 100k, better than every other model at the larger size.

Memory span (P1): A ridge decoder from the KC code at character t recovers character $t - k$ in 99.9% of cases for $k = 0$, 90–92% for $k = 1$, 57–60% for $k = 2$, 35–38% for $k = 3$, 24–26% for $k = 4$ and 20–21% for $k = 5$, approaching

TABLE II
VALIDATION BPC ON THE THREE 20K/5K SEGMENTS (OFFSETS 0, 300K, 600K). BRAIN: SPARSE 160 PN, CLIPPED KC FEATURES.

Model	0	300k	600k
Hashed 3-character context head	3.205	3.115	2.970
+ brain	3.100	3.013	2.858
Kneser-Ney 3-gram	2.968	2.957	2.817
Kneser-Ney 5-gram	2.676	2.588	2.475
+ brain	2.734	2.651	2.511
Kneser-Ney 7-gram	2.670	2.570	2.466
GRU (hidden 256)	2.857	2.985	2.637

the rate of always guessing the most frequent character (15–16%) by $k \approx 7$ (Fig. 4, left). The brain thus holds about four characters, fading fast, which is less context than a 5-gram uses. This also explains the learning curve: with little data the hashed trigram table is sparse and the brain’s smooth code of the last few characters generalises better; with more data the table fills in.

Is the real wiring needed? (P3): We built two rewired connectomes by permuting (presynaptic neuron, weight) pairs among edges: over the whole brain, or only within blocks of presynaptic and postsynaptic cell class (KC, PN, MBON, dopaminergic neurons, APL, otherwise the annotated super-class). Every neuron keeps its in-degree, its out-degree and the sign and weights of its outputs; duplicate edges created by the permutation were merged (1.5% and 0.3% of edges). With classes kept, PN→KC connections remain but go to random KCs; the KC code keeps the same memory span as in the real connectome (1.00, 0.95, 0.62, 0.38, 0.25 for $k = 0$ –4) and, like it, adds nothing on top of a 5-gram (its KC features cost $+0.09$ to $+0.12$ BPC there, against $+0.04$ to $+0.06$ for the real connectome). With the whole brain rewired, KCs no longer receive PN input and stay silent, yet the whole-brain readout (context head plus all features) scores 3.122, 3.037 and 2.874 BPC against 3.106, 3.024 and 2.846 for the real connectome, within the readout-fit noise; the class-preserving rewiring gives 3.118, 3.056 and 2.880. The information comes from other neurons that also hold a short fading trace. One rewiring of each kind was tested.

Taken together: with this input protocol and a linear readout, the frozen connectome behaves as a generic fading memory of about four characters. Its advantage over the hashed context head was real but came from a weak baseline, and it does not depend on the specific wiring.

VIII. A WRITE GATE FOR THE MUSHROOM BODY (PILOT)

A GRU learns what to store and when [10]; the frozen brain cannot. The mushroom body can write, and the companion study found that the dopaminergic teacher chooses where a memory is stored [7]. We therefore tested whether a content-dependent teacher lets the mushroom body learn next-character associations online, as flies learn an odour in one or a few trials.

Design: Fifteen dopaminergic cell types whose gating patterns over the MBONs overlap little ($|\cos| < 0.5$) and

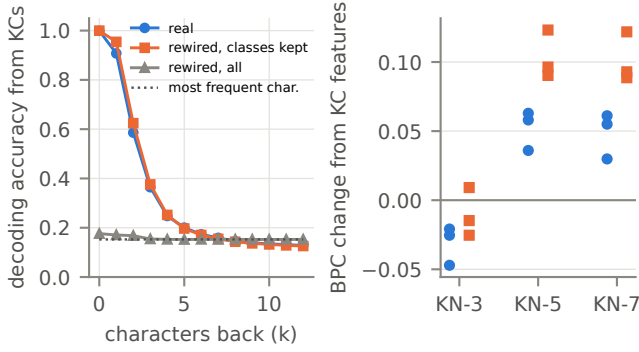


Fig. 4. Left: decoding the character k steps back from the KC code of the real connectome, a connectome rewired within cell classes, and a fully rewired one (mean of three segments). Right: change in BPC when KC features are added to Kneser-Ney models (points: segments; colours as left).

whose pulse delivers a substantial gate serve as fifteen character classes. When character $t + 1$ arrives, the type of its class is pulsed, and the KC $\rightarrow$ MBON update writes the KC eligibility trace of the preceding characters (not the current one): recent context $\rightarrow$ class of what came next. The prediction for character $t + 1$ is read without training: the memory current evoked by the KCs active at character t is decomposed by least squares into the gate patterns of the classes (memory currents superpose linearly [7]), and the class with the largest coefficient is chosen. The text consists of five made-up five-letter words with disjoint letters in random order, so that within a word the next letter is predictable only after the word has been seen. Controls: a teacher that pulses a random class (the same number and strength of writes, no content), and no writes. The criterion, fixed before the runs, was that accuracy on repeated occurrences exceed both first occurrences and the random teacher.

Result: With the content teacher, accuracy rose from 4% on first occurrences to 22% on second and 27% on later ones (mean of five seeds; balanced over classes 26%, range 15–36%), while the random teacher stayed at chance (balanced 7.8% against 6.7% for $1/K$; Fig. 5). One correction was made after the first runs: two-cell dopaminergic types whose pulse delivered almost no gate produced huge least-squares coefficients from noise and dominated the predictions in one of three seeds; we then kept only types delivering a total gate of at least 1 and reran all seeds, adding two new ones. The mushroom body can thus learn associations online when the teacher carries content. Whether this adds anything to a strong n -gram on real text is a separate question (below).

Real text: We then ran the same content-taught mushroom body over the three TinyShakespeare segments (25k characters each, plasticity on throughout; the 65 characters assigned to the 15 classes round-robin in order of frequency) and, as a control, with the random teacher. The memory did learn: its read-out class prediction was right for 11–14% of characters with the content teacher against 5–6% with the random teacher (balanced 8.5–9.0% against 6.5–7.4%),

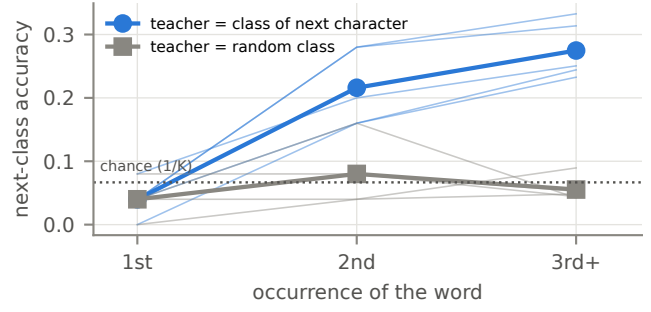


Fig. 5. Online learning in the mushroom body with a content-dependent dopaminergic teacher: accuracy of the next character’s class by occurrence of the word (thin lines: five seeds; thick: mean). Dotted: chance.

though below always guessing the most frequent class (16–18%). The second criterion, fixed before the run, was that its memory features (the memory current on the 97 MBONs and its class decomposition) add to a Kneser-Ney 5-gram. They did not: stacked on the 5-gram they changed BPC by +0.003 to +0.018 with the content teacher and +0.005 to +0.019 with the random teacher. A causal cache of recent character statistics did no better (-0.006 to $+0.012$). On this text, what an online memory of the last hundred or so characters can learn, the 5-gram already knows.

IX. PLAYGROUND

To make these results tangible we wrote GPF, the “Generative Pretrained Fly”, a terminal application (distributed with the repository) that continues a prompt with a chosen model, GPT-style. The name is a joke: the connectome’s only pretraining is evolution, and only its linear readout was trained, on 20k characters. The prompt is fed one character at a time, then the model samples a character, feeds it back and continues. The connectome model simulates all 166,700 neurons for each character (about 20 characters per second on a laptop GPU). With the prompt “ROMEO:” at temperature 0.7 the connectome model writes Shakespeare-shaped text that is mostly not words (“I’ the as fragent Roman now I / thrirlywerGere: yetty truduse”), a 5-gram trained on the same 20k characters writes slightly better nonsense, a 7-gram on a million characters writes plausible fragments (“O, ’twas us / fellow of the young Princes have the triumph?”), and a GRU on a million characters writes the most coherent text (“Shall I put was the bastards prepare to make the state and a far thoughts”).

X. DISCUSSION

For its original goal this project ends with a clear negative result. A frozen reservoir with a linear readout can only expose what its dynamics retain [11], and the fly connectome driven one character at a time retains about four characters, fading fast, which is no more than a 5-gram knows and no more than a rewired network of the same degrees retains. The improvements found along the way were real but measured against

a weak baseline: first against nothing (the earlier benchmark was an n-gram score), then against a hashed context table that a standard smoothed n-gram beats by 0.5 BPC. The lesson we draw is methodological: every gain must be measured against the strongest simple baseline available, and against a control that removes the property being credited (here, the wiring).

What the mechanism study adds is an account of why, and a direction. Dense input erased the sparse KC code; a frozen network holds only a fading trace; and plasticity with a teacher that is the same for every character writes nothing item-specific. The pilot shows the other side: when the dopaminergic teacher depends on content, as a GRU’s update gate does, the mushroom body learns associations after one or two exposures. This is where the fly’s circuit differs from a generic reservoir. On real text the same mechanism learned too, but what it learned in its short window a 5-gram already knew. Whether a richer teacher, longer eligibility traces or slower decay would let it store something n-gram statistics do not have remains open; we have not shown that it can.

Limitations: One connectome and one rewiring of each kind; one corpus; short splits (20k characters, one 100k run); one readout fit per case; a linear readout; KN and GRU baselines without extensive tuning; the exploratory table was tuned on the validation split (the later analyses were not); in the language-model runs only 0.5% of KCs fired per character and no MBON fired, a quiet regime set by the input drive, and other input protocols might retain more; the pilot used a synthetic task and a decoder correction made after the first runs.

XI. CONCLUSION IN PLAIN TERMS

We asked whether the wiring diagram of a fruit-fly brain, simulated but never trained, could help predict the next letter of Shakespeare. At first it seemed to: the model guessed about 43% of letters correctly. On checking, almost all of that came from a simple lookup table of what usually follows the previous three letters, which scores as well without any brain. Adding the fly’s own learning mechanism wrote information into the brain but did not help, and working out why became a separate study.

That study showed that we had been feeding the brain too much input at once, so that nearly all of its Kenyon cells, the neurons the fly uses to tell smells apart, lit up for every letter. With sparser input the brain did beat our lookup table, but our lookup table was a poor one. A standard, well-smoothed table of five-letter sequences does much better than any of our brain models, and adding the brain to it only makes things worse. The reason is simple: the simulated brain remembers roughly the last four letters, and a five-letter table already knows those. A brain whose wiring we scrambled, keeping only how many connections each neuron has, remembers just as much.

The fly’s brain can do one thing a fixed lookup table cannot: learn on the spot. When we let its dopamine neurons signal which letter came next, the way a reward or a punishment signals what an odour meant, the mushroom body started predicting letters of made-up words after seeing them once

or twice. On real Shakespeare it learned as well, but nothing that the five-letter table did not already know.

The practical lessons are to compare every result with the strongest simple baseline available, to test whether the thing given credit (here, the real wiring) is actually needed, and to audit results before building on them: here the audits found a simulator bug, an inflated benchmark, a weak baseline and a preprocessing artefact.

USE OF AI TOOLS

This is independent research by a high-school student. The author directed Phases 1–2 and proposed the plasticity route (see Roles); from Phase 3 on, much of the investigation was carried out by AI agents under the author’s instructions. AI agents (OpenAI Codex, DeepSeek via OpenCode, Anthropic Claude) implemented and ran training scripts, parameter sweeps, parallel runs, audits, and this report’s analysis and drafting.

DATA AND CODE

<https://github.com/LxCenady/Fly-Is-All-You-Need>.

REFERENCES

- [1] S. Berg *et al.*, “Sexual dimorphism in the complete *Drosophila* male central nervous system connectome,” *Cell*, vol. 189, no. 18, pp. 5504–5526.e15, 2026.
- [2] P. K. Shiu *et al.*, “A *Drosophila* computational brain model reveals sensorimotor processing,” *Nature*, vol. 634, pp. 210–219, 2024.
- [3] D. Costi, A. Hadjiivanov, D. Dold, J. Hale, and D. Izzo, “The *Drosophila* connectome as a computational reservoir for time-series prediction,” *Biomimetics*, vol. 10, no. 5, 341, 2025.
- [4] leetae9yu, “fly-as-a-lm,” GitHub repository, <https://github.com/leetae9yu/fly-as-a-lm>, accessed 2026-09-28.
- [5] S. Dasgupta, C. F. Stevens, and S. Navlakha, “A neural algorithm for a fundamental computing problem,” *Science*, vol. 358, no. 6364, pp. 793–796, 2017.
- [6] Y. Shen, S. Dasgupta, and S. Navlakha, “Reducing catastrophic forgetting with associative learning: a lesson from fruit flies,” *Neural Comput.*, vol. 35, no. 11, pp. 1797–1819, 2023.
- [7] Q. Li, “Three addresses of a connectome-constrained mushroom-body memory: dopamine chooses the compartment, sparse Kenyon-cell codes set the cue, side- and subtype-specific routing shapes the output,” 2026. Mechanism study that grew out of the present project.
- [8] R. Kneser and H. Ney, “Improved backing-off for M-gram language modeling,” in *Proc. IEEE ICASSP*, 1995, pp. 181–184.
- [9] S. F. Chen and J. Goodman, “An empirical study of smoothing techniques for language modeling,” *Comput. Speech Lang.*, vol. 13, no. 4, pp. 359–394, 1999.
- [10] K. Cho *et al.*, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” in *Proc. EMNLP*, 2014, pp. 1724–1734.
- [11] M. Lukoševičius and H. Jaeger, “Reservoir computing approaches to recurrent neural network training,” *Comput. Sci. Rev.*, vol. 3, no. 3, pp. 127–149, 2009.